

C++第四章

1. More on Functions

- Function object

- 在 C++ 里一切都需要有类型，因此函数也必须有类型。
- 函数对象（`std::function`）就是定义函数类型的工具。
- 函数的输入和输出类型（签名）就决定了它的类型。

示例代码：

```
1 #include <functional>
2
3 function<double(double)>;           // 表示一个接收 double, 返回 double 的函数
4 function<void(int, vector<int>&)>; // 表示接收 int 和 vector<int>&, 返回
void 的函数
```

- 作用：

1. 可以把函数当作参数传递（pass function as input parameter）。
2. 可以把函数当作变量保存和操作（define function as a variable）。

- Function object - example: pass as input parameter

- 解释了如何把函数对象当作参数传递给另一个函数。
- 示例代码：

```
1 #include <functional>
2 function<double(double, double)> payoff;
```

定义了一个函数类型，接收两个 `double`，返回一个 `double`。

- 举例：

```
1 double payoff_call(double S, double K) { return max(S - K, 0.0); }
2 double payoff_put(double S, double K) { return max(K - S, 0.0); }
3
4 double apply_payoff(double S, double K,
5                     function<double(double, double)> payoff) {
6     return payoff(S, K);
7 }
8
9 cout << apply_payoff(100, 120, payoff_call) << '\n'; // 0
10 cout << apply_payoff(100, 120, payoff_put) << '\n'; // 20
```

`apply_payoff` 接收一个函数对象作为参数，可以灵活调用不同的 `payoff` 计算方式。

- Lambda function - example

- 普通函数 不能在函数内部定义，否则会报错。

```

1 int main() {
2     double payoff_call(double S, double K) { ... } // 错误
3 }

```

- 但是 **Lambda 函数可以在函数内部定义**：

```

1 int main() {
2     const function<double(double, double)> payoff_call =
3         [](double S, double K) { return max(S - K, 0.0); };
4
5     const auto payoff_put =
6         [](double S, double K) { return max(K - S, 0.0); };
7 }

```

`[]` 表示 Lambda 的开始，后面跟参数和函数体。

Lambda functions - syntax

Lambda 函数的语法形式是：

```

1 [ capture clause ] (parameters) modifier -> return type {
2     definition of body
3 }

```

- **capture clause**：定义要捕获的外部变量。
- **parameters**：函数参数。
- **modifier**（可选）：高级用法，常省略。
- **return type**（可选）：可以自动推断。
- **body**：函数体，和普通函数类似。
- **Lambda functions - syntax**
 - 展示了如何把 **普通函数** 转换成 **Lambda 函数**。

- 普通函数写法：

```

1 double payoff_call(double S, double K) { return max(S-K, 0.0); }

```

- Lambda 函数写法：

```

1 auto payoff_call = [](double S, double K) -> double { return
    max(S-K, 0.0); };

```

- 说明：

- `[]` 表示 **捕获列表**（capture clause）。
- `(double S, double K)` 是参数。
- `-> double` 是返回类型（可选，编译器通常能推断）。
- `{ return ... }` 是函数体。

◦ Capture clause (捕获列表)

- 当 Lambda 需要访问外部变量时，需要通过 **捕获列表** 来引入。
- 例子 1：按值捕获（只读，不修改外部变量）。

```
1 int abc = 123;
2 auto larger_than_abc = [abc](int x) { return x > abc; };
3 cout << larger_than_abc(124) << '\n'; // 输出 1 (true)
```

- 例子 2：按引用捕获（可以修改外部变量）。

```
1 auto larger_and_replace_abc = [&abc](int x) {
2     if (x > abc) abc = x;
3 };
4 larger_and_replace_abc(125);
5 cout << abc << '\n'; // 输出 125
```

◦ 捕获方式总结：

- `[]`：空，不捕获任何变量。
- `[name1, name2, ...]`：按值捕获指定变量。
- `[&]`：按引用捕获所有变量。
- `[=]`：按值捕获所有变量（在创建 Lambda 时拷贝一份）。
- `[name1, &name2]`：混合方式，name1 按值捕获，name2 按引用捕获。
- `[name1 = expression]`：用表达式初始化，然后按值捕获。

◦ 示例：

```
1 int x = 1;
2 auto func = [x2 = x + 1]() { cout << x2 << '\n'; };
3 func(); // 输出 2
```

这里展示了 **初始化捕获 (C++14 引入)** 的写法，可以在捕获时对变量进行初始化。

• C++ 函数绑定 (`std::bind`)

`bind` 的作用：`bind` 可以调整函数的输入参数，生成一个新的函数对象。

有两个常见用途：

1. 偏函数 (Partial function) / 柯里化 (Curried function)

- 可以先给函数的一部分参数赋固定值，生成一个新的函数，新的函数只需要输入剩下的参数。
- 例子：

```
1 f = f(a, b, c);
2 g = bind(f, a, b, _1);
3 // g(x) = f(a, b, x)
```

2. 调整参数顺序

- 可以用 `bind` 改变输入参数的排列顺序。

```
1 g = bind(f, _2, _1);
2 // g(a, b) = f(b, a)
```

偏函数 (Partial function)

给定函数:

```
1 auto greater(int a, int b) { return a > b; }
```

如果我们想要一个“大于某个固定值 X”的函数, 可以:

1. 用 **lambda**:

```
1 auto greater_than_x = [x](int b) { return greater(b, x); };
```

2. 用 **bind** + 占位符 `_1, _2`:

```
1 using namespace std::placeholders;
2 auto greater_than_x = bind(greater, _1, x);
```

示例 `count_if` 函数:

```
1 int count_if(const vector<int>& v, function<bool(int)> f) {
2     int count = 0;
3     for (size_t i = 0; i < v.size(); ++i) {
4         if (f(v[i])) count++;
5     }
6     return count;
7 }
```

用法:

```
1 vector<int> vec{1, 2, 3, 4};
2 cout << count_if(vec, bind(greater, _1, 3)) << '\n'; // 统计 >3 的数量
3 cout << count_if(vec, bind(greater, 3, _1)) << '\n'; // 统计 <3 的数量
```

参数重排 (Re-arranged function)

使用 `bind` 可以交换输入参数的位置。

```
1 bool greater(int a, int b) { return a > b; }
2 auto lesser = bind(greater, _2, _1); // lesser(a, b) = greater(b, a)
```

另一个例子: 期权定价函数

```
1 double payoff_call(double S, double K) {
2     return max(S - K, 0.0);
3 }
4 auto payoff_put = bind(payoff_call, _2, _1);
5 // 把输入顺序调换: payoff_put(K, S) = payoff_call(S, K)
```

• Recursive functions

- 介绍 **递归函数** 的定义: 函数可以调用自身。

- 递归适用于能被分解为更小子问题的场景。
- 需要有 **终止条件**，否则会无限递归。
- 例子：**阶乘 factorial**

```
1 int factorial(int n) {  
2     if (n < 0) throw logic_error("n must be non-negative");  
3     if (n == 0) return 1;  
4     return n * factorial(n - 1);  
5 }
```

- 提醒：要注意 **整数溢出**，比如 factorial 增长很快，int 容量有限。

Recursive function - Fibonacci

- 例子：**斐波那契数列** (1,1,2,3,5,8,13,...) 。
- 递归版本：

```
1 long long fib_recursive(int n) {  
2     if (n <= 1) return 1;  
3     return fib_recursive(n - 1) + fib_recursive(n - 2);  
4 }
```

- 这个版本虽然直观，但存在 **效率问题**。

Recursive function - Fibonacci with memorization

- 在之前的 **递归版本** 中有大量重复计算。
- 这里展示了 **递归 + 记忆化 (memoization)** 的方法：
 - 使用 `vector` 存储已经计算过的结果。
 - 每次递归调用前先检查缓存，若已有值则直接返回。
- 代码亮点：
 - 用 `function` 定义递归 lambda (必须显式捕获自己: `&fib_internal`) 。
 - 使用 `NOT_INCLUDE (-1)` 作为未计算标记。
- 优点：避免重复计算；把指数级复杂度降为 **线性 $O(n)$** 。

• The pros and cons of recursion

优点 (Pros):

1. 表达方式优雅，形式化好（数学风格简洁）。
2. 发现递归模式会让人有种“哇”的感觉。
3. 很多算法测试喜欢递归写法。

缺点 (Cons):

1. 执行开销大：函数调用次数多，每次有栈帧成本。
2. 有重复计算时，时间复杂度会指数爆炸。
3. 如果可以，尽量避免递归，改写成线性迭代。
4. 如果必须递归，建议结合 **记忆化 memoization**，存储中间结果以提升效率。

2. C++ STL (标准模板库)

- **Software time complexity: big O notation**
 - **时间复杂度 (time complexity analysis)** : 之前已经测试过算法运行时间, 现在开始用理论的方式分析算法复杂度。
 - **空间复杂度 (space complexity)** : 和时间复杂度类似, 但内存通常不是主要限制, 因此这里重点讲时间复杂度。
 - **大 O 表示法 (Big O notation)** : 用来定量描述算法复杂度, 标注 **最坏情况 (upper limit, worst case)** 的运行时间。
- **Types of time complexity** (常见时间复杂度类型)
 - **Constant $O(1)$**
常数时间, 执行时间与输入规模无关。例如判断 `x % 2 == 0`。
 - **Linear $O(N)$**
线性时间, 必须遍历整个输入。例如数组遍历。
 - **Quadratic $O(N^2)$**
二次方复杂度, 输入规模加倍时运行时间变 4 倍。例如双重循环。
 - **Factorial $O(N!)$**
阶乘复杂度, 通常是穷举算法, 增长极快, 实际不可用。
 - **Logarithmic $O(\log N)$**
对数复杂度, 随着输入增长, 运行时间增长得很慢。常见于 **分治算法**、**二分查找**。

 image-20250824002307021

- $O(1)$
 - 查找 **数组/向量中第 n 个元素** 是 $O(1)$, 因为可以直接通过偏移量找到, 不随数据规模增加而增加运算次数。
 - 例子: 高斯求和公式, 把 $1+2+3+\dots+n$ 转换为 $n(n+1)/2$, 从 **$O(N)$** 变成 **$O(1)$** 。
 - 注意: 虽然是 $O(1)$, 但常数因子可能很大, 在实际中也会影响速度。
- $O(N)$
 - 示例: **线性查找 (Linear Search)**
遍历 vector 中的每个元素, 逐个比较是否等于目标值。
 - 代码:

```
1 bool linear_search(const vector<Point>& a, const Point& key) {
2     for (size_t i = 0; i < a.size(); ++i) {
3         if (a[i].x == key.x && a[i].y == key.y)
4             return true;
5     }
6     return false;
7 }
```
 - 时间复杂度 $O(N)$, 因为要遍历所有元素。
 - 提到 **并行化**: 在多线程环境下, $O(N)$ 可以优化成 $O(N/p)$, 极端情况下接近 $O(1)$, 但要考虑通信开销。
- $O(\log N)$
 - 示例: **二分查找 (Binary Search)**, 适用于排好序的数组/vector。

- 每次比较后把搜索范围缩小一半，最坏情况需要 $\log_2(N)$ 步。
- 代码：

```

1  bool binary_search(const vector<int>& a, int key) {
2      auto low = 0, high = a.size() - 1;
3      while (low <= high) {
4          const auto mid = midpoint(low, high); // C++20
5          if (a[mid] < key) {
6              low = mid + 1;
7          } else if (a[mid] > key) {
8              high = mid - 1;
9          } else {
10             return true;
11         }
12     }
13     return false;
14 }

```

- 二分查找是 **分治算法 (Divide-and-Conquer)** 的典型例子。
- $O(N^2)$
 - 当我们要找 **向量中两个元素之间的最短距离** 时，需要比较每一对元素。
 - 这需要大约 $N(N-1)/2$ 次比较，时间复杂度是 $O(N^2)$ 。
 - 示例代码：

```

1  void shortest_distance(std::vector<int>& a) {
2      int shortest = 0;
3      for (size_t i = 1; i < a.size(); ++i) {
4          for (size_t j = i+1; j < a.size(); ++j) {
5              shortest = min(shortest, abs(a[i] - a[j]));
6          }
7      }
8  }

```

- 注意：有时候可以优化，比如如果要想找最长距离，只需比较 **最大值和最小值**，就能在 $O(N)$ 时间内完成。
- Bubble sort vs Quicksort
 - **冒泡排序 (Bubble sort):**
 - 时间复杂度：最坏情况下是 $O(N^2)$ 。
 - 对小规模数据 (<100) 或几乎有序的数据，冒泡排序表现还行。
 - 思想：
 - 一次次遍历数组，每次比较相邻两个数，如果顺序错了就交换。
 - 像气泡一样，大的元素会逐渐“冒”到数组末尾。
 - 时间复杂度：最坏 $O(N^2)$ ，最好 $O(N)$ (数据本来有序时)。
 - 空间复杂度： $O(1)$ ，因为是原地排序。
 - **快速排序 (Quicksort):**
 - 平均复杂度 $O(N \log N)$ ，最坏情况下 $O(N^2)$ 。
 - 是 **分治算法 (Divide and Conquer)** 的典型代表：

- 选一个元素作为 pivot。
 - 把数组分成两部分：小于 pivot 的一部分，大于 pivot 的一部分。
 - 递归排序两部分。
 - 思想：
 - 分治法 (Divide and Conquer) 的典型应用。
 - 过程：
 1. 选一个基准值 (pivot)，一般选数组中间或第一个。
 2. 把比 pivot 小的元素放在左边，比 pivot 大的放在右边。
 3. 对左右子数组递归执行同样的操作。
 - 时间复杂度：平均 $O(N \log N)$ ，最坏 $O(N^2)$ (当数组接近有序且 pivot 选择不当时)。
 - 空间复杂度： $O(\log N)$ (递归栈)。
- Summary

- 时间复杂度分析的意义：

- 用来量化算法性能，找出瓶颈，避免低效算法。
- 不同算法复杂度差别很大，应选择合适的。

- 注意：理论上的时间复杂度 ≠ 实际性能，实际应用中要测试比较。

- `std::array` / `std::vector` / `std::list`

- C++ STL 提供了不同的容器：

- `std::array`
- `std::vector`
- `std::list`

- 它们在性能上有差异：

- 速度：`array > vector > list`
- 使用场景：
 - `array` 固定大小，速度快。
 - `vector` 动态数组，性能和灵活性折中，默认推荐。
 - `list` 链表，适合频繁插入删除，但访问性能差。

- 使用方法：`#include <array>`，`#include <vector>`，`#include <list>`

- Comparison of array/vector/list

- Vector

- 动态可调整大小 (resizable)。
- 插入操作复杂度： $O(N+M)$ 。
- 尾部追加 (append)： $O(1)$ 。
- 随机访问： $O(1)$ 。

- Array

- 和 `vector` 类似，但大小在编译期确定 (fixed size)。
- 大小必须是常量或字面值。
- 大部分操作是 $O(1)$ 。

- **List**

- 可调整大小。
- 在任意位置插入或删除: $O(1)$ 。
- 随机访问: $O(N)$ (因为需要逐个遍历)。
- 插入/删除不需要移动其它元素。

- **std::list**

- list 元素存储在 **双向链表 (double-linked chain)** 中。
- 可以前后两个方向遍历。
- 适合频繁插入和删除操作的场景, 因为不需要移动其他元素。
- 内存不需要连续 (不像 vector 那样需要连续内存)。
- 图示:

- 访问一个元素, 需要从头或尾开始逐个走过去。
- 插入/删除操作只需调整指针, 效率高。

- **Time complexity for list**

- **读写首尾元素**: $O(1)$ 。
- **插入/删除**: $O(1)$ 。
- **访问中间元素**: $O(N)$ (因为要遍历)。
- 示例应用:
 - 在交易所订单簿 (order book) 中, 删除订单非常常见 (研究显示 >90% 的订单会被取消)。
 - order book 通常是顺序执行, 不需要随机访问。
 - 所以 list 很适合用来实现订单簿。

- **Operations with list L**

- **获取大小**: `L.size()`
- **访问头元素**: `L.front()`
- **访问尾元素**: `L.back()`
- **在头部插入/删除**: `L.push_front(e)` / `L.pop_front()`
- **在尾部插入/删除**: `L.push_back(e)` / `L.pop_back()`
- **删除所有等于某个值的元素**: `ListName.remove(e)`
- **拼接两个 list**: `L1.splice(L1.end(), L2)`

- 结果: L1 包含所有元素, L2 被清空。

- **`list::splice` 的作用**

- `splice` 并不是“复制”元素, 而是**把节点从一个链表移动到另一个链表**。
- list 是 **双向链表**, 节点的内存独立存在, 每个节点有前后指针。
- `splice` 只是改动节点指针的连接, 把 L2 的节点全部挂接到 L1 的尾部。

- 为什么 L2 会被清空?**

- `splice` 之后, L2 的所有节点都被“摘走”挂接到 L1。
- 因为节点已经完全转移, L2 不再拥有这些节点, 所以它变成空的。

- 注意：这里没有任何元素被复制或销毁，仅仅是“指针的重连”，所以效率是 $O(1)$ （只改指针）。

- **std::array**

- `std::array` 是 **固定大小的数组**（大小在编译时决定，不能改变）。
- 主要功能就是存储和访问元素。
- 访问和写入操作都是 $O(1)$ ，即常数时间。
- 本质上和 `std::vector` 类似，但大小固定。

图中展示了数组在内存中的布局：元素是连续存储的，通过 **起始地址 + 偏移量** 定位。

Example - creation and access

- 代码演示了 `std::array` 的声明与初始化：

```
1 array<int, 3> arr{};           // 初始化为 {0,0,0}
2 array<int, 3> arr2{3, 3, 3};   // 初始化为 {3,3,3}
```

- **边界检查：**

- `arr[2]` 访问没有边界检查，越界访问可能产生错误结果，但编译不会报错。
- `arr.at(i)` 会进行运行时检查，越界会抛出异常。

Store a matrix in an array - 1

- 讲解了如何用数组存储 **矩阵**。
- 内存是线性的（扁平的），所以二维矩阵要映射到一维数组。
- 对于矩阵元素 $A_{i,j}$ ，对应到一维数组下标公式是：

$$A[(i - 1) * n + (j - 1)]$$

- 也可以直接用二维数组来表示：

```
1 array<array<int, 2>, 2> A;
2 A[0][0] = 234;
3 cout << A[0][0];
```

- **std::vector**

- `std::vector` 是 **动态数组**，可以在运行时调整大小。
- 元素存储在 **连续的内存位置**。
- 连续内存的优势：
 - 提高 CPU cache 命中率（cache locality）。
 - 因此 `vector` 的访问性能优于 `list`。
- `list` 的数据节点可能分散在内存中，所以 **没有 cache locality**。

std::vector - how it grows - 1

- 当 `vector` 调用 `push_back()` 或 `insert()` 需要增加元素，但内存不够时，会触发 **扩容**。
- 扩容机制：
 1. `vector` 会分配一块更大的连续内存。
 2. 把已有的所有元素复制/移动到新的内存块里。
 3. 释放原来的内存。

- 图中展示了 `vector` 内存不够时，如何搬移到新的内存区域以继续增长。
- 扩容代价很高（需要分配新内存并复制已有数据），所以要 **避免频繁扩容**。

建议方法：

- 如果能预估数据量大小，**一次性创建足够大的 `vector`**，避免之后改变大小。
- 如果无法预估大小，可以用以下两种方式：
 - `reserve(size_t n)`：预先分配内存容量（capacity 增加，但 size 不变），再用 `push_back()` 插入元素。
 - `size()` 只表示当前已有元素数目，不会因为 `reserve()` 改变。
 - 可用 `capacity()` 检查预分配的内存大小。
 - `resize(size_t n)`：直接改变 `vector` 的大小，新加的元素可以用 `[]` 访问。

Time complexity of vector

- 读取/写入元素**： $O(1)$
- `push_back()`：均摊时间复杂度 $O(1)$ （因为扩容不频繁）。
- 删除元素** (`erase`)： $O(N)$ ，因为要移动后续元素。
- 插入元素** (`insert`)： $O(N+M)$ ，其中 N = 插入的新元素个数， M = 需要移动的元素个数。

Useful functions for std::vector

常见函数及作用：

函数	用途
<code>at(n)</code>	带边界检查的访问元素
<code>size()</code>	获取 <code>vector</code> 的大小
<code>resize(n)</code>	改变 <code>vector</code> 的大小（可能丢失尾部元素）
<code>clear()</code>	删除所有元素
<code>insert()</code>	在指定位置插入元素
<code>erase()</code>	删除指定位置的元素
<code>pop_back()</code>	删除最后一个元素
<code>push_back(val)</code>	在尾部追加元素
<code>emplace()</code>	就地构造元素
<code>emplace_back()</code>	就地构造并追加到尾部
<code>== / !=</code>	比较两个 <code>vector</code> 是否相等

• `emplace / emplace_back`

- `emplace_back` 和 `emplace` 与 `push_back`、`insert` 类似，但更高效。
- 区别在于：
 - `push_back`：先创建一个临时对象，再拷贝到 `vector` 中。

- `emplace_back`: 直接在 `vector` 内部原地构造对象 (in-place), 避免了拷贝。

◦ 示例代码:

```
1 vector<vector<int>> vv;
2 vv.push_back(vector<int>(3, 10)); // 先生成临时 vector<int>(3,10),
   再拷贝
3 vv.emplace_back(3, 10); // 直接在 vv 内部生成, 无拷贝
```

• Vector: example 1

◦ 最基本的 `vector` 用法: 定义并访问元素。

```
1 vector<double> v1(2);
2 v1[0] = 1.0;
3 v1[1] = 2.0;
4 cout << v1[0] << ", " << v1[1] << '\n'; // 输出 1,2
```

• Vector: example 2

◦ 讲解了 **不要混用初始化方式**:

- 如果创建空的 `vector`, 用 `push_back()` 增加元素;
- 如果创建时已经指定大小 (`vector<int> A(50)`), 应该用 `[]` 来修改元素。

◦ 错误示例:

```
1 vector<int> A(50); // 已经有 50 个 0
2 for (size_t i = 0; i < 50; ++i) {
3     A.push_back(3); // 又增加了 50 个 3
4 }
5 // 结果: A 一共有 100 个元素, 而不是 50 个
```

• Vector: example 3

- 创建一个包含 **100 个随机数** 的 `vector`。
- 用 `resize(100)` 先确定大小, 再用循环赋值。
- 示例代码:

```
1 random_device rd{};
2 auto mtgen = mt19937{rd};
3 auto urdd = uniform_real_distribution<>(2.0, 5.0);
4
5 vector<int> v_rand;
6 v_rand.resize(100);
7
8 for (auto &v : v_rand) {
9     v = urdd(mtgen); // 生成随机数赋值
10 }
11
12 cout << v_rand[34] << '\n'; // 打印第 35 个数
```

• STL Iterators - introduction

- **迭代器**是 STL 提供了一种统一访问容器中元素的方法。
- 以前我们在 `vector` 和 `array` 中用 `[]` 下标来遍历, 因为它们支持随机访问。

- 但对于 `list`, `set`, `map` 这种**没有下标**的容器, 就需要用迭代器。

- **迭代器 = 一个可以遍历容器的“指针对象”。**

- **STL Iterators - definition**

- 迭代器定义在 `<iterator>` 头文件里 (一般 `vector` 等容器头文件已经包含)。

- 定义迭代器的方法:

```
1 container<type>::iterator it;
```

- 更常见的写法是用 `auto`:

```
1 auto it = v.begin();
```

- **Iterator - example 1**

- 示例:

```
1 vector<int> v{1, 2, 3};
2 auto it = v.begin();
3 cout << *it << '\n'; // 输出 1
```

- `v.begin()`: 返回指向第一个元素的迭代器。

- `v.end()`: 返回指向“最后一个元素的下一个位置”的迭代器 (不是最后一个元素)。

- 迭代器存的值是**位置**, 需要用 `*it` 取值。

- **v.begin() and v.end()**

- `v.end()` **不是最后一个元素**, 而是“尾后迭代器”。

- 遍历时要用:

```
1 for (auto it = v.begin(); it != v.end(); ++it) {
2     cout << *it << " ";
3 }
```

- `v.begin()` 和 `v.end()` 定义了区间 `[first, last)` (前闭后开)。

- 常用快捷函数:

- `v.front()` → 第一个元素 (等价于 `*v.begin()`)

- `v.back()` → 最后一个元素 (等价于 `*(--v.end())`)

- **Question**

- **不同容器的迭代器功能不同:**

- `vector` / `array` 的迭代器是 **随机访问迭代器 (Random Access Iterator)**, 可以做 `it + 1`、`it - 2` 等操作。

- `list` 的迭代器是 **双向迭代器 (Bidirectional Iterator)**, 只能用 `++it` 和 `--it`。

- **推荐通用做法:** 使用

- `next(it, n)` / `prev(it, n)` → 返回新的迭代器

- `advance(it, n)` → 直接修改迭代器位置

- **Iterator - move it around**

提供几种通用的迭代器移动方法：

1. `advance(it, n)` → 修改迭代器，让它前进或后退 n 步 (n 可为负数)。
2. `next(it, n)` / `prev(it, n)` → 返回新的迭代器，不改变原迭代器。
3. `distance(it1, it2)` → 计算两个迭代器之间的距离 (元素个数)。

注意：所有迭代器操作都没有边界检查，超出范围会导致错误。

• Iterator - example

代码示例：

```
1 auto it = v.begin();
2 it++; // 移动到第二个元素
3 cout << *it; // 输出第三个元素
4
5 cout << *(prev(v.end(), 1)); // 输出最后一个元素
```

- `++it` → 前进一个位置。
- `prev(v.end(), 1)` → 返回倒数第一个元素 (`end()` 是尾后迭代器)。
- 如果越界 (如访问 `v.end()`)，结果是未定义的。

• Usage of iterator: insertion / erase for vector

演示如何用迭代器在 `vector` 里插入/删除：

- `v.insert(pos, value)` → 在迭代器 `pos` 指定位置前插入 `value`。
- `v.erase(pos)` → 删除 `pos` 指向的元素。

示例：

```
1 vec.insert(vec.begin(), 3); // 在开头插入 3
2 vec.insert(vec.end(), 4); // 在末尾插入 4
3 vec.erase(vec.begin() + 1); // 删除第二个元素
4 vec.insert(vec.begin(), vec.begin(), vec.end()); // 复制整个自己
```

`insert(pos, first, last)` 的语义是：

把区间 `[first, last)` 中的元素，按顺序插入到位置 `pos` 之前。

- `pos = vec.begin()`：插入到最前面；
- `first = vec.begin(), last = vec.end()`：要插入的区间就是 `vec` 目前的所有元素。

• Iterator for iteration

- 在 C++11 之前，通常用显式迭代器来遍历容器：

```
1 for (vector<int>::iterator iter = v.begin(); iter != v.end(); ++iter)
2 {
3     // ...
4 }
```

- C++11 引入了 `auto` 关键字，可以简化写法：

```
1 for (auto iter = v.begin(); iter != v.end(); ++iter) {
2     // ...
3 }
```

- Use range-for from C++11

- C++11 引入了 **范围 for 循环 (range-for)**，让遍历容器更简洁：

```
1 vector<int> abc{1, 2, 3};
2 for (auto v : abc) { v++; }           // 遍历元素副本，修改不影响原容器
3 for (auto &v : abc) { v++; }          // 遍历元素引用，可以修改原容器
4 for (const auto &v : abc) { v++; }    // 错误: const 引用禁止修改
```

- range-for for convenient iteration

- 展示如何用 range-for 简单打印 `vector` 里的所有元素：

```
1 void print_vec(const vector<int> &v, string name) {
2     cout << name << " : ";
3     for (auto x : v) { cout << x << ", "; }
4     cout << '\n';
5 }
6
7 vector<int> abc{1, 2, 3};
8 print_vec(abc, "abc");
```

- What's the difference between different iteration methods?

- **索引方式 (index-based)**：只适用于 `array` 和 `vector` 这样的随机访问容器。

```
1 for (size_t i = 0; i < v.size(); ++i) { v[i] += 123; }
```

- **迭代器方式 (iterator-based)**：适用于所有容器（包括 `list`, `set`, `map` 等）。

```
1 list<int> lx{1, 2, 3};
2 for (auto it = lx.begin(); it != lx.end(); ++it) {
3     cout << *it << ", ";
4 }
```

- **范围 for (range-for)**：语法更简洁，适用于所有容器。

```
1 for (auto v : lx) { cout << v << ", "; }
```

- 如何在使用 range-for 或 iterator 遍历容器时同时追踪索引 (index)

方法 1：使用一个追踪变量

手动维护一个索引变量 `i`，每次循环时自增：

```
1 int i = 0;
2 list<int> lx{1, 2, 3};
3 for (auto v : lx) {
4     cout << "[" << i << " ] " << v << ", ";
5     ++i;
6 }
```

输出结果类似：

```
1 | [0] 1, [1] 2, [2] 3,
```

适合所有容器，简单直接。

方法 2: 使用 `std::distance`

`std::distance(it1, it2)` 计算两个迭代器之间的距离，也就是相当于“索引值”。

但是要注意：

- 对于 **vector**（随机访问迭代器），`distance` 是 **$O(1)$** 。
- 对于 **list**（双向迭代器），`distance` 是 **$O(n)$** ，性能较差。

例子：

```
1 | list<int> Lx{1, 2, 3};  
2 | cout << distance(Lx.begin(), Lx.end()); // 输出 3
```